

Agile Testing Plan

Testing, Roles, Techniques & Principles

Agile Software Development | Dr. Hitesh Mohapatra

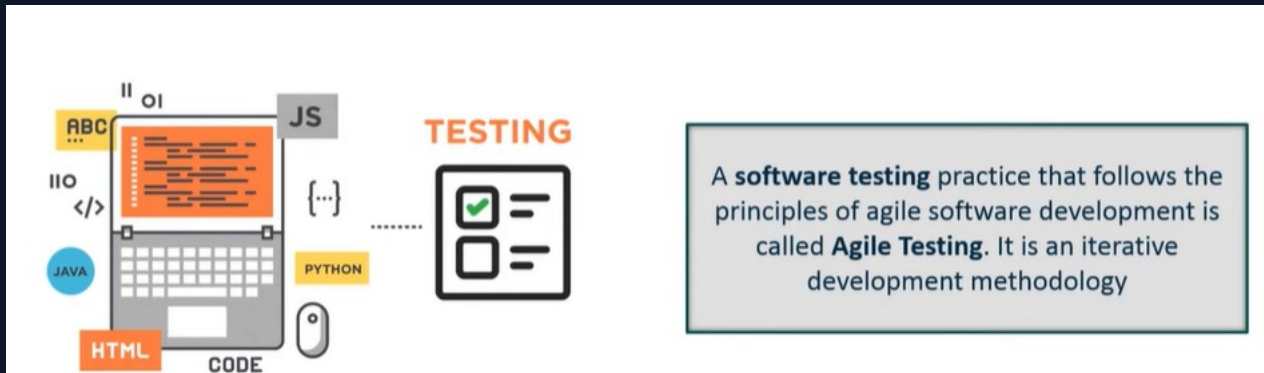
Learning Outcomes

- Explain how Agile test planning connects user stories to execution
- Map Agile roles to testing responsibilities and collaboration
- Choose key techniques aligned with Agile principles
- Design a living test plan for continuous feedback

What is an Agile Testing Plan?

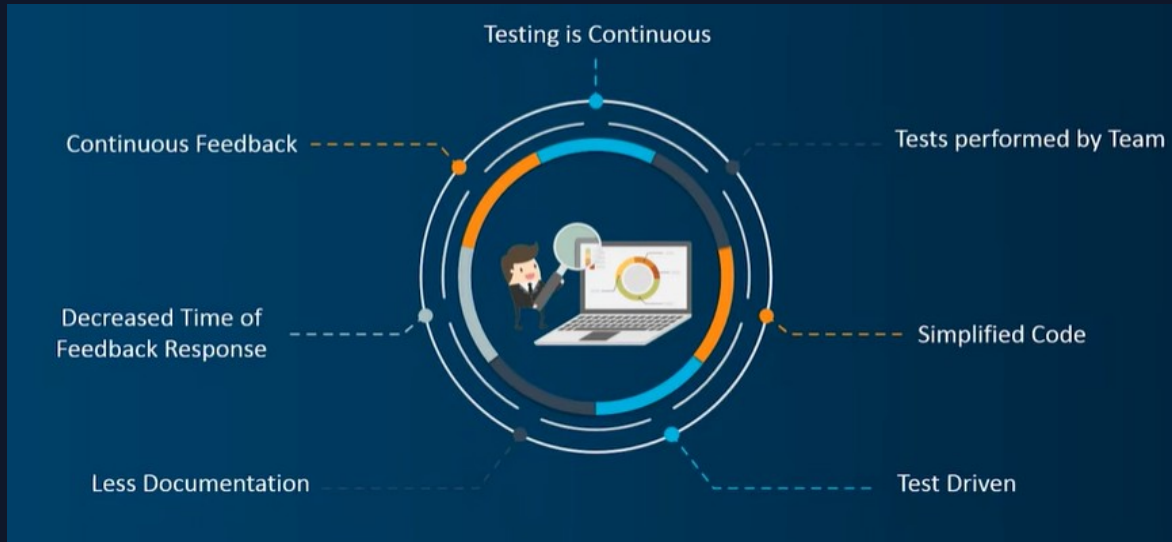
Not a frozen document—a lightweight, evolving agreement that links **what to test**, **who ensures quality**, and **how quality is built in** across every sprint.

- Covers: scope, schedule, resources, environments, defect management, exit criteria
- Adapted each sprint based on story changes and risks
- Focused on user value, not process overhead



Core Principles (Part 1)

- **Iterative & Continuous:** Testing in every sprint, not after development
- **Flexibility (Living Plan):** Updated when stories, priorities, or risks change
- **Collaboration:** Testing is whole-team activity; shared ownership of quality



Core Principles (Part 2)

- **User-Story Alignment:** Test scope driven by user stories and acceptance criteria
- **Early Defect Discovery:** Find issues fast to reduce late rework
- **Quality as Responsibility:** Developers, testers, and PO all drive quality

Agile Roles & Testing Responsibilities

Product Owner

Clarifies acceptance criteria; validates outcomes; decides story acceptance/rejection

Scrum Master

Facilitates practices; removes impediments; supports team collaboration

Agile Roles & Testing (continued)

Developer

Collaborates on acceptance criteria; supports defect resolution; improves testability

Tester / QA

Designs tests; performs functional & non-functional testing; communicates quality risks

AGILE TESTING METHOD

- Behavior Driven Development (BDD)
- Acceptance Test Driven Development(ATDD)
- Exploratory Testing

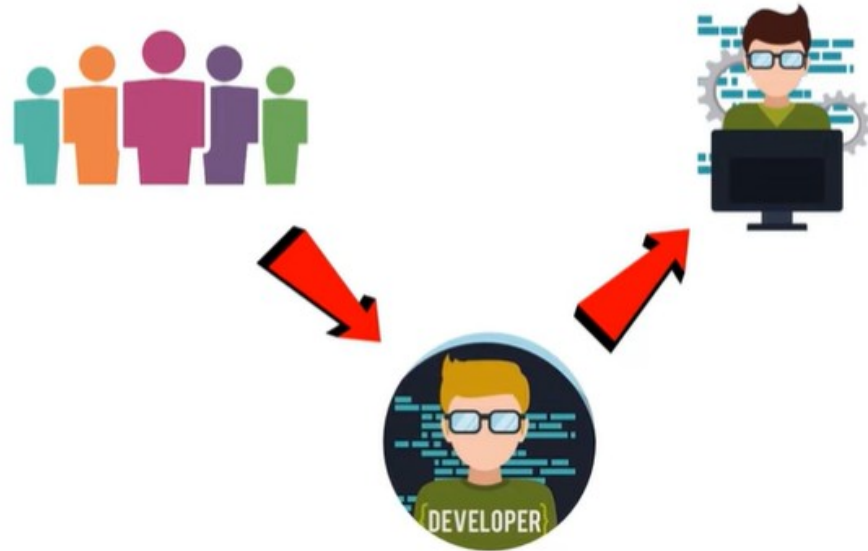
Behavior Driven Development (BDD)



AGILE TESTING METHOD

- Behavior Driven Development (BDD)
- Acceptance Test Driven Development(ATDD)
- Exploratory Testing

Acceptance Test Driven Development(ATDD)



AGILE TESTING METHOD

- Behavior Driven Development (BDD)
- Acceptance Test Driven Development(ATDD)
- Exploratory Testing

Exploratory Testing



Key Testing Techniques

- **Story/Acceptance-Criteria-Driven:** Convert user story criteria into checks (Given–When–Then)
- **Exploratory Testing:** Time-boxed exploration to discover unexpected behaviors and usability issues
- **Automation-First:** Identify repeatable paths; automate regression; evolve each sprint

Key Techniques (continued)

- **Risk-Based Focus:** Prioritize deep testing on critical areas (payments, auth, safety)
- **Defect Management:** Fast feedback loops with clear logging, tracking, and resolution steps
- **ATDD / Acceptance Test-Driven:** Tests defined before coding to drive development

Agile Testing Life Cycle



Example: "Apply Coupon" Feature

User Story: As a customer, I want to apply a coupon to my cart so I receive a discount.

- **Acceptance Criteria:** Coupon applied → discount shown; expired coupon → error message
- **Sprint Timeline:** 2 weeks; release next sprint
- **Environments:** Dev, staging, production-like test

Acceptance Checks (Example)

- **Check 1:** Given cart total is ₹1000, when coupon SAVE10 is applied, then discount is 10% and final total is ₹900
- **Check 2:** Given coupon is expired, when applied, then user sees error and total unchanged
- **Check 3:** Given minimum cart value is ₹500 and cart is ₹400, coupon cannot be applied

Exploratory Testing Charter (Example)

Goal: Discover edge cases and usability issues in coupon application

- Multiple coupons on one order (stacking rules)
- Rounding and tax interaction with discount
- Concurrent application (two tabs open)
- Poor network and timeout handling

Automation & Risk Focus (Example)

Automate (Regression)

Success path, expired coupon, boundary totals

Priority (Risk)

Correctness, audit logs, security against abuse

Test Plan Key Components

- **Scope:** User stories & acceptance criteria (not full requirements)
- **Schedule:** Sprint timeline + testing phases within each sprint
- **Resources:** Team roles, skill levels, automation tools, test environments
- **Risks:** High-priority testing areas, known constraints

AGILE TEST PLAN

01

Testing Scope

New Functionalities

02

03

Types of testing based on the features complexity

Load & Performance Testing

04

05

Infrastructure Consideration

Mitigation or Risk Plans

06

07

Resourcing

Deliverables & Milestones

08

Sprint Exit Criteria

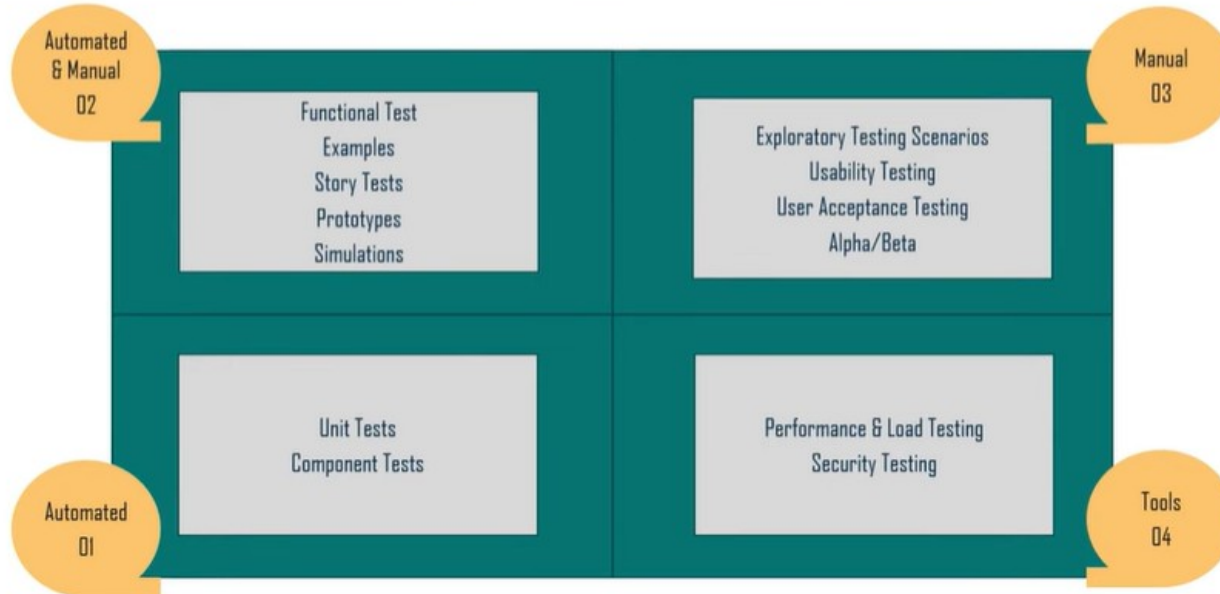
- All critical acceptance criteria pass
- No open critical/blocker defects
- Smoke & regression suite green in CI/CD
- Known risks documented for PO decision
- Code review + test coverage standards met

How They Link Together

Principles guide what we test → **Roles** define who tests → **Techniques** enable how we test

- **Iterative principle** → Daily testing in sprint (tester role)
- **Collaboration principle** → PO clarifies criteria, dev & tester pair on edge cases
- **Flexibility principle** → Plan changes sprint-to-sprint; adapt techniques based on feedback

AGILE TESTING QUADRANTS



Best Practices Checklist

- ✓ Test early (in sprint, not after sprint)
- ✓ Automate stable regression paths; manual for exploration
- ✓ Collaborate on acceptance criteria before dev starts
- ✓ Communicate risks and blockers daily
- ✓ Keep defect feedback loops <24 hours

Common Pitfalls to Avoid

- **Testing at the end:** Test in sprint; don't wait for "test phase"
- **Unclear acceptance criteria:** PO & tester clarify before coding starts
- **No automation:** Manual testing doesn't scale; automate regressions
- **Siloed roles:** Tester != QA ninja; whole team owns quality

Tools & Frameworks

- **BDD Frameworks:** Cucumber, SpecFlow (Given–When–Then syntax)
- **Automation:** Selenium, TestNG, JUnit, Cypress
- **CI/CD:** Jenkins, GitHub Actions, GitLab CI
- **Test Management:** JIRA, TestRail, Zephyr

Key Takeaways

- Agile test plans are **living**, not frozen; adapted each sprint
- **Principles, Roles, & Techniques** are tightly linked
- Testing is a **team activity**; whole team owns quality
- Test **early & often** to reduce surprises and rework

Thank You!

Questions?

Agile Testing Plan | Dr. Hitesh Mohapatra